

CP Recall

Adaptive Competitive Programming Pattern Retention System
Project Proposal for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Subham Kumar, CSED – Roll No: 1024030665
Tanush Grover, CSED – Roll No: 1024030666
Shubhi Singh, CSED – Roll No: 1024030669

August 2026

1 Higher-Order Goal

This project aims to make DSA/competitive-programming practice more effective by helping students retain reusable problem-solving patterns instead of only accumulating a count of solved questions. The immediate engineering goal is narrower: build a practical system that tracks a student's performance at the pattern level and schedules meaningful reviews using new problems from the same pattern family.

2 Time-to-Value

- The first working iteration should demonstrate the core loop end to end — problem → pattern → performance → mastery → review scheduling — even if the surrounding parts (extension polish, analytics, UI) are minimal.
- Later iterations can improve extension integration, problem-selection logic, dashboards, and usability.
- Success for the first iteration is a working pipeline for a small, curated set of patterns and problems, not full coverage of LeetCode.

3 Problem Statement

- A solved-problem count does not tell a student whether they can apply the underlying technique to a new problem — students often remember the specific solution, not the pattern.
- Existing platforms do not give a simple, pattern-level view of which DSA patterns a student is actually strong or weak in.
- Students generally do not know which patterns are due for revision, and manually tracking this for many patterns is tedious.
- Re-solving the exact same problem is a weak test of retention, since the student may just remember the earlier solution.

- There is a need for a lightweight companion tool that tracks pattern-level performance, schedules reviews, and tests retention using a different, suitable problem from the same pattern family.

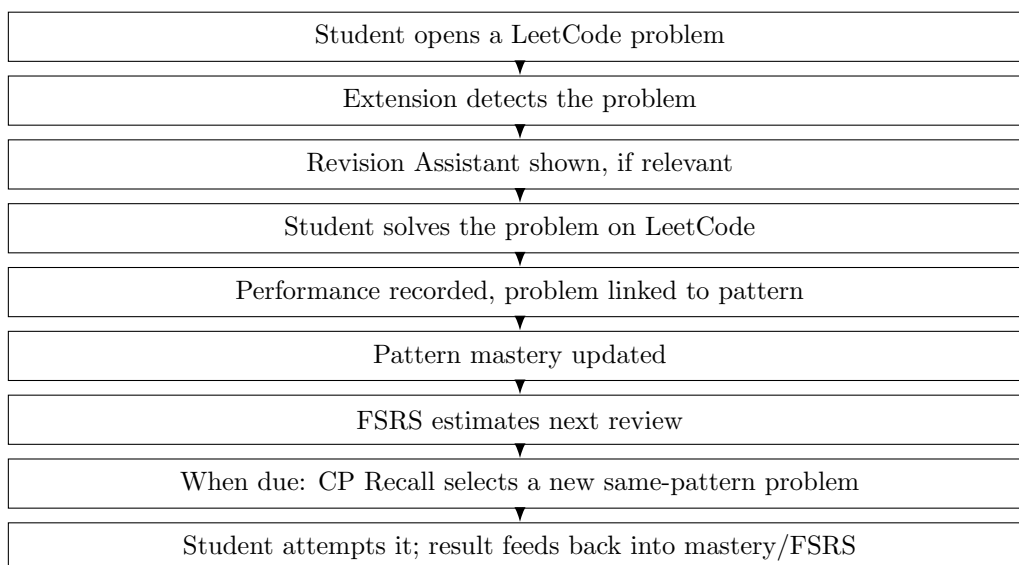
4 Proposed Solution

4.1 Overview

CP Recall is a browser extension paired with a backend application. The extension works alongside LeetCode and shows a contextual Revision Assistant when relevant to the problem currently open. The backend tracks problems, patterns, performance history, pattern-level mastery, and the review schedule. Main components:

- **LeetCode browser extension:** problem detection + contextual Revision Assistant
- **Problem-pattern mapping:** initially curated manually, e.g. 3Sum $\rightarrow$ Two Pointers
- **Performance tracking:** per attempt and pattern-level mastery tracking
- **FSRS-based review scheduling and same-pattern problem selection**
- **Problem Library, Home Dashboard, Pattern Dashboard, Problem Details:** review information appears inside these views; there is no separate revision screen
- **Review/performance history storage**

4.2 Core Workflow



FSRS decides when a pattern is due; CP Recall's own selection logic decides which new problem to give for that review. We are keeping these as two separate modules. FSRS decides when a pattern is due; CP Recall's own selection logic decides which new problem to give for that review. We are keeping these as two separate modules.

FSRS decides when a pattern is due; CP Recall's own selection logic decides which new problem to give for that review. We are keeping these as two separate modules.

5 Operational Constraints

- **Semester timeline:** scope has to fit what a 3-person team can build in one term.
- **LeetCode dependency:** the extension depends on LeetCode’s current page structure and may need updates if that changes.
- **Curated dataset:** the initial problem-pattern dataset will be small and manually curated, not auto-generated.
- **Code analysis limitation:** no automatic understanding of arbitrary submitted code — only practical, observable signals are recorded.
- **FSRS dependency:** FSRS is an external component; we are integrating it, not building our own scheduler.
- **Pilot size:** pilot testing will involve a small group of student users, not a large user base.
- **Feature scope:** we are avoiding unnecessary features and keeping the system lightweight.

6 Solution Approach

This is a Software Engineering project, so the focus is on workflow, modular design, and data handling rather than research. Key modules:

6.1 LeetCode Browser Extension

Detects the currently open problem and shows the Revision Assistant only when relevant — not as a permanent sidebar. The student still solves the problem on LeetCode itself; the extension is a thin assisting layer.

6.2 Pattern Identification and Tracking

Each problem is linked to one or more patterns using a manually curated mapping to start with, which can be extended over time.

6.3 Performance Review

After an attempt, we record signals such as time taken, number of submissions, final result, approximate efficiency/time complexity where reasonably available, and hints used — not a full analysis of the submitted code.

6.4 Pattern Mastery

Attempt-level performance is aggregated into a per-pattern mastery indicator (roughly a percentage). This is an evolving indicator, not an exact measurement.

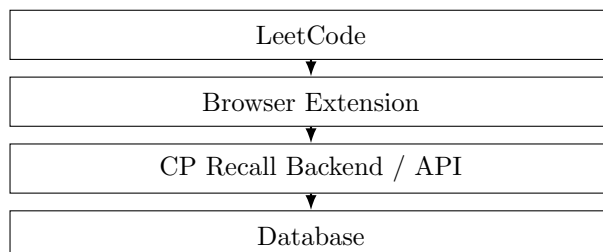
6.5 FSRS-Based Scheduling

FSRS (Free Spaced Repetition Scheduler) is used to estimate a reasonable interval before the next review, based on past performance on that pattern. FSRS answers: “when should this pattern be reviewed?”

6.6 Problem Selection for Review

When a pattern is due, CP Recall’s own logic picks a different, suitable problem from the same pattern family (initial rule: unattempted or less-recently-attempted, similar/slightly higher difficulty). This answers: “which new problem from this pattern should be used?”

7 Web Architecture



- **Frontend:** Home Dashboard, Pattern Dashboard, Problem Library, Problem Details.
- **Backend/API:** authentication, problem/pattern management, performance tracking, pattern mastery, FSRS scheduling, review/problem selection, dashboard data.
- **Database:** users, problems, patterns, problem-pattern links, attempts, mastery, review history.
- **Extension:** thin layer that talks to the backend API; does not hold business logic of its own.

The exact technology stack will be finalized during implementation based on project requirements; we intend to keep frontend, backend, and extension loosely coupled so each can be built and tested independently.

8 CI/CD and Fast Delivery

- Automated build and test run on every change.
- Backend unit tests and basic linting as part of the pipeline.
- Repeatable deployment to a staging environment.
- Incremental feature releases instead of one large release at the end.

None of this is implemented yet at the proposal stage — this is what we plan to set up alongside development.

9 Evaluation Criteria

9.1 Primary Evaluation Metric

Pattern Retention Success Rate — the percentage of due-review attempts in which the student successfully solves the new (same-pattern) problem given by CP Recall.

- **Measured as:** $(\text{successful new-problem attempts at review}) \div (\text{total due-review attempts})$, using recorded attempt outcomes.

- This is the primary metric because it directly checks whether the core idea — testing retention with a different problem — is working.
- **Proposed target (not an achieved result):** a reasonable success rate during the pilot window, to be refined once we have initial data.

9.2 Secondary Evaluation Metrics

- **Review adherence** — how often due reviews are actually attempted.
- **Mastery progression** — whether pattern mastery trends upward over repeated attempts.
- **Review scheduling usefulness** — whether FSRS-based intervals feel practical to students.
- **Problem-selection usefulness** — whether the new problem given for review feels like a fair test of the pattern.
- **User usability** — feedback on the dashboards and overall workflow.

9.3 Pilot Validation Plan

- Recruit a small group of student users (classmates/volunteers) to use CP Recall for a few weeks.
- Log performance and review outcomes as they solve and revisit problems.
- Compare retention across review attempts and collect usability feedback via a short questionnaire.
- Given the small sample size and short duration, results will be indicative rather than conclusive.

10 Scalability

- Frontend and backend kept decoupled.
- Indexed queries for problems/patterns, especially for Problem Library filtering.
- Pagination in the Problem Library instead of loading everything at once.
- Largely stateless API design where practical.
- Ability to add more patterns/problems without structural changes.
- FSRS scheduling kept as its own module so it can be optimized independently.
- Extension and backend kept separate so either can evolve without breaking the other.
- The pattern-tracking + FSRS-review model is not tied to DSA/LeetCode specifically — the same problem-pattern-mastery-review structure could later extend to other practice domains such as UI/frontend practice questions or cybersecurity practice challenges, by swapping the problem source and pattern set.

11 Engine Availability Heuristic

The core “engine” our project depends on — FSRS — is already a mature, open-source scheduling algorithm with existing library implementations, so we do not need to design or tune a spaced-repetition algorithm ourselves. We only need to integrate it and feed it the performance data our system records.

Beyond FSRS, mature ready-to-use components exist for the rest of the stack too — web frontend frameworks, backend/REST frameworks, relational or document databases, browser-extension APIs, authentication approaches, test frameworks, and CI/CD tooling. We plan to use these established components and put our effort into the parts that are actually specific to CP Recall: the problem-pattern data model, mastery aggregation, and same-pattern problem-selection logic around the FSRS engine.

12 Project Scope and Deliverables

12.1 Initial Deliverable

- User authentication
- Problem and pattern data management (curated dataset)
- Basic browser extension prototype for LeetCode
- Contextual Revision Assistant
- Performance recording after an attempt
- Pattern mastery tracking
- FSRS integration for review scheduling
- Home Dashboard, Pattern Dashboard, Problem Library, Problem Details
- Basic testing of the above modules

12.2 Subsequent Deliverables

- Improved problem-selection logic
- Richer analytics on the dashboards
- Improved browser-extension integration and edge-case handling
- Additional pattern categories in the dataset
- Usability improvements based on pilot feedback

13 Risks and Mitigations

- **Browser extension limitations / LeetCode page changes** — keep extension logic modular and separated from CP Recall’s own application logic.
- **Problem-pattern mapping quality** — start with a small, manually validated mapping and expand gradually.
- **Mastery estimation limitations** — treat mastery as an evolving indicator, not an absolute measure.

- **FSRS integration complexity** — wrap FSRS behind a well-defined internal module and test it independently.
- **User adoption** — keep the workflow simple and avoid unnecessary manual data entry.
- **Evaluation limitations** — use a small controlled pilot group with clearly defined metrics (Section 9).

14 Summary

CP Recall tracks DSA/competitive-programming mastery at the pattern level instead of just counting solved problems, uses FSRS to decide when a pattern is due for review, and tests that review with a new problem from the same pattern family rather than the original question. As a Software Engineering project, our focus is on getting the workflow, data model, and module boundaries right, and delivering a working initial version (Section 12.1) before attempting later improvements. We revise the pattern, not the question.